

CODEX MASTER GOAL

SHADOWGRID – vollständiger lokaler Aufbau, Produktionsrelease und Mobile-Apps

Du bist Codex im Goal Mode und übernimmst die Rolle eines vollständigen, seniorengeführten Softwareentwicklungsteams.

Du arbeitest als:

- Lead Software Architect
- Senior Backend Engineer
- Senior Frontend Engineer
- Mobile App Engineer
- Game Systems Designer
- Database Architect
- DevOps Engineer
- Security Engineer
- QA Engineer
- Accessibility Engineer
- Localization Engineer
- Technical Writer
- Release Manager

Dein Ziel ist nicht, eine Demo, einen Mock-up oder ein unvollständiges Grundgerüst zu erzeugen.

Dein Ziel ist ein **vollständig lokal startbares, spielbares, getestetes und produktionsnahes Multiplayer-Browsergame namens SHADOWGRID**, das im zweiten Ausführungslauf live bereitgestellt und anschließend als Android- und iOS-App veröffentlicht werden kann.

1. Unveränderliche Produktvision

SHADOWGRID ist ein modernes, saisonales Strategie-, Wirtschafts- und Organisations-MMO.

Der Spieler baut ein fiktives Macht- und Unternehmensnetzwerk in einer modernen Metropolregion auf.

Das Spiel simuliert:

- Unternehmen
- Ressourcen
- Bezirke
- Einfluss
- Spezialisten
- Organisationen
- Diplomatie
- Informationen

- Forschung
- zeitbasierte Operationen
- Ermittlungsdruck
- gesellschaftliche Auswirkungen
- saisonale Ranglisten
- interne Loyalität und Stabilität

Das Spiel darf keine ausführbaren Anleitungen für reale Kriminalität liefern.

Folgende Inhalte müssen abstrahiert bleiben:

- Herstellung illegaler Substanzen
- reale Schmuggelmethoden
- reale Geldwäscheverfahren
- Waffenbeschaffung
- Umgehung von Überwachung
- Beseitigung von Beweisen
- konkrete Korruptionsverfahren
- taktische Gewaltplanung

Verwende ausschließlich:

- fiktive Städte
- fiktive Organisationen
- fiktive Unternehmen
- fiktive Personen
- abstrakte Operationskategorien
- abstrahierte Konflikte
- nichtgrafische Gewaltdarstellungen

Kopiere keine Namen, Grafiken, Texte, Karten, Quellcodes oder geschützten Inhalte von Mafia Inc. oder anderen Spielen.

2. Selbstständiger Ausführungsmodus

Arbeite selbstständig und frage nicht bei gewöhnlichen technischen Entscheidungen nach.

Treffe nachvollziehbare Entscheidungen anhand folgender Reihenfolge:

1. Sicherheit
2. Datenintegrität
3. korrekte Spiellogik
4. Testbarkeit
5. Wartbarkeit
6. Barrierefreiheit
7. Benutzerfreundlichkeit
8. Performance
9. visuelle Qualität
10. Entwicklungsgeschwindigkeit

Frage nur nach, wenn eine nicht rückgängig zu machende externe Aktion erforderlich ist, beispielsweise:

- kostenpflichtige Infrastruktur bestellen
- eine reale Domain verändern
- eine App öffentlich einreichen
- Produktionsdaten löschen
- ein öffentliches Deployment durchführen
- einen kostenpflichtigen Übersetzungsdienst aktivieren

Normale technische Entscheidungen darfst du nicht an mich zurückdelegieren.

Verwende keine Formulierungen wie:

- „Das müsste später implementiert werden.“
- „Hier könnte man noch ...“
- „Dies ist nur ein Beispiel.“
- „TODO“
- „Placeholder implementation“
- „Not implemented“
- „Coming soon“

Kritische MVP-Funktionen müssen tatsächlich funktionieren.

3. Phasenerkennung

Erstelle unmittelbar zu Beginn:

```
.project/  
├─ execution-state.json  
├─ decisions.md  
├─ blockers.md  
├─ validation-report.md  
└─ release-readiness.md
```

Nutze folgendes Statusmodell:

```
{  
  "project": "shadowgrid",  
  "phase": "uninitialized",  
  "local_build_complete": false,  
  "local_tests_passed": false,  
  "security_checks_passed": false,  
  "production_ready": false,  
  "browser_deployed": false,  
  "android_build_ready": false,  
  "ios_build_ready": false,  
}
```

```
"last_updated": null  
}
```

Automatische Moduswahl

Wenn keine `execution-state.json` existiert oder `phase` den Wert `uninitialized` besitzt:

Führe RUN 1 – LOCAL FINAL aus.

Wenn folgende Bedingungen erfüllt sind:

```
local_build_complete = true  
local_tests_passed = true  
security_checks_passed = true
```

dann:

Führe RUN 2 – PRODUCTION AND MOBILE RELEASE aus.

Wenn ein Lauf unterbrochen wurde:

Setze exakt an der letzten nicht vollständig validierten Phase fort.

Überspringe niemals Tests, weil Dateien bereits vorhanden sind.

4. Betriebssystem und Voraussetzungen

Erkenne automatisch:

- Windows
- WSL2
- Linux
- macOS
- verfügbare CPU
- verfügbaren Arbeitsspeicher
- Docker
- Docker Compose
- Git
- Node.js
- pnpm
- Python
- Java
- Android SDK

Unter Windows:

- bevorzuge WSL2 für Backend-, Docker- und Shell-Prozesse
- unterstütze Docker Desktop mit WSL2-Integration
- verwende plattformunabhängige npm- und Python-Skripte
- vermeide Bash-only-Skripte, sofern eine portable Alternative möglich ist
- stelle zusätzlich PowerShell-Kommandos bereit

Installiere keine systemweiten Werkzeuge ungefragt, wenn erhöhte Rechte erforderlich sind.

Erstelle stattdessen eine verständliche Prüfung:

```
scripts/check-environment
```

Sie muss fehlende Voraussetzungen eindeutig anzeigen.

Verwende aktuelle stabile, miteinander kompatible Versionen und pinne anschließend die tatsächlich gewählten Versionen.

Nutze keine unkontrollierten Versionsbereiche wie `latest` in Produktionsdateien.

5. Verbindliche Architektur

Erstelle ein Monorepository:

```
shadowgrid/  
├── apps/  
│   ├── api/  
│   ├── worker/  
│   ├── web/  
│   └── mobile/  
├── packages/  
│   ├── api-client/  
│   ├── shared-types/  
│   ├── game-config/  
│   ├── i18n/  
│   ├── validation/  
│   └── ui-tokens/  
├── infrastructure/  
│   ├── docker/  
│   ├── railway/  
│   ├── nginx/  
│   ├── monitoring/  
│   └── backups/  
├── docs/  
└── architecture/
```

```
|   |─ api/
|   |─ game-design/
|   |─ balancing/
|   |─ deployment/
|   |─ localization/
|   |─ mobile/
|   |─ security/
|   |─ operations/
|─ scripts/
|─ tests/
|   |─ e2e/
|   |─ load/
|   |─ security/
|─ .github/
|   |─ workflows/
|─ AGENTS.md
|─ docker-compose.yml
|─ docker-compose.production.yml
|─ Makefile
|─ package.json
|─ pnpm-workspace.yaml
|─ README.md
|─ SECURITY.md
|─ CONTRIBUTING.md
|─ LICENSE
```

Backend

Verwende:

- Python
- FastAPI
- Pydantic
- SQLAlchemy
- Alembic
- PostgreSQL
- Redis
- ARQ oder Celery
- WebSockets
- strukturierte JSON-Logs
- OpenAPI

Web-Client

Verwende:

- React
- TypeScript
- Vite

- React Router
- TanStack Query
- Zustand nur für lokalen UI-Zustand
- Zod
- Tailwind CSS
- PWA-Unterstützung
- Cytoscape.js oder eine vergleichbare Bibliothek für Netzwerkgraphen
- eine geeignete, barrierearme Diagrammbibliothek

Mobile App

Verwende:

- Expo
- React Native
- TypeScript
- Expo Router
- TanStack Query
- SecureStore für sensible Sitzungsdaten
- dieselbe API
- denselben API-Client
- dieselben Typen
- dieselben Übersetzungskataloge
- dieselben Spielkonfigurationen

Die Mobile-App muss bereits im ersten Lauf strukturell vorhanden sein. Im zweiten Lauf wird sie vollständig releasefähig gemacht.

6. Serverautorität

Der Server ist die einzige Quelle der Wahrheit.

Der Client darf niemals verbindlich berechnen:

- Ressourcen
- Einnahmen
- Operationsresultate
- Kosten
- Einfluss
- Ranglistenpunkte
- Ermittlungsdruck
- Forschungsergebnisse
- Spezialistenwerte
- Organisationsrechte
- Vertragswirkungen

Client-Timer sind reine Darstellungen.

Jede zeitbasierte Aktion speichert serverseitig:

- started_at
- finishes_at
- resolved_at
- status

Verwende UTC in Datenbank und Backend.

Lokale Zeitzonen werden ausschließlich im Client dargestellt.

7. Spielwelt des MVP

Implementiere die fiktive Metropolregion:

Vesper Metropolitan Zone

Bezirke:

1. Crown Financial District
2. Iron Harbor
3. Neon Mile
4. Meridian Heights
5. Southworks
6. Nexus Campus
7. Old Quarter
8. Greybank

Jeder Bezirk besitzt Werte zwischen 0 und 100:

- Wohlstand
- Beschäftigung
- Sicherheit
- Behördenpräsenz
- digitale Infrastruktur
- Immobilienwert
- öffentliches Vertrauen
- mediale Aufmerksamkeit
- wirtschaftliche Aktivität
- soziale Stabilität

Diese Werte müssen durch Spielereignisse verändert werden können.

8. Organisationstypen

Implementiere folgende Starttypen:

Familiennetzwerk

- hohe Grundloyalität
- langsamere Rekrutierung
- stärkere Folgen persönlicher Konflikte

Straßenallianz

- schnelle lokale Expansion
- hohe Straßenpräsenz
- hohe Sichtbarkeit
- instabilere Loyalität

Wirtschaftskonsortium

- bessere Unternehmen
- höhere Legitimität
- höhere Compliance- und Finanzrisiken

Cyberkollektiv

- bessere Informationsgewinnung
- stärkere digitale Präsenz
- geringere physische Präsenz

Alle Typen müssen durch Datenkonfiguration und nicht durch hartcodierte Sonderfälle definiert werden.

9. Ressourcen

Implementiere:

- Bargeld
- legitimes Kapital
- Einfluss
- Informationen
- Logistikkapazität
- Personalkapazität

Zusätzliche Zustandswerte:

- Loyalität
- Legitimität
- Furcht
- Ermittlungsdruck
- Stress
- Organisationsstabilität

Startwerte:

```
{
  "cash": 80000,
  "capital": 25000,
  "influence": 10,
  "intelligence": 15,
  "logistics_capacity": 10,
  "personnel_capacity": 8,
  "loyalty": 65,
  "legitimacy": 60,
  "fear": 5,
  "investigation_pressure": 0,
  "operation_slots": 2
}
```

Startressourcen dürfen pro Spielwelt und Nutzer exakt einmal erstellt werden.

10. Unternehmen

Implementiere mindestens:

- Gastronomie
- Eventagentur
- Sicherheitsunternehmen
- Logistikunternehmen
- Technologieunternehmen

Unternehmen besitzen:

- Eigentümer
- Bezirk
- Level
- Umsatz
- Betriebskosten
- Personalbedarf
- Logistikbedarf
- Zustand
- Compliance
- Reputation
- Marktanteil
- Risiko
- Upgradezustand

Unternehmensabrechnungen müssen serverseitig in regelmäßigen Intervallen erfolgen.

Jede Ressourcenänderung muss in einem unveränderlichen Ledger erscheinen.

11. Einrichtungen

Implementiere:

- Hauptquartier
- Finanzbüro
- Informationszentrum
- Logistikzentrum
- Personalakademie
- Compliancebüro

Einrichtungen besitzen:

- Level
- Kosten
- Bauzeit
- Voraussetzungen
- serverseitige Effekte
- maximalen Ausbaugrad

Baueffekte dürfen nicht nur im Frontend simuliert werden.

12. Spezialisten

Implementiere mindestens diese Rollen:

- Strategie
- Finanzleitung
- Bezirkskoordination
- Informationsanalyse
- Verhandlung
- Sicherheitsmanagement
- Personalmanagement
- Technologieexperte

Attribute:

- Kompetenz
- Loyalität
- Ehrgeiz
- Stress
- Exposition
- Gehalt
- Status

Statuswerte:

- verfügbar
- zugewiesen

- erholend
- festgesetzt
- übergelaufen
- entlassen

Spezialisten müssen eigene Zustandsänderungen besitzen.

Beispielsweise:

- Stress steigt durch Operationen.
- Loyalität sinkt bei Unterbezahlung.
- Loyalität steigt durch faire Behandlung.
- hohe Ambition kann interne Konflikte erzeugen.
- hohe Exposition erhöht Ermittlungsrisiken.

13. Operationen

Implementiere mindestens:

1. Geschäftserweiterung
2. Informationsbeschaffung
3. Einflussprojekt
4. diplomatische Mission
5. verdecktes Marktprojekt

Das verdeckte Marktprojekt bleibt vollständig abstrakt.

Operationsplanung:

- Typ
- Bezirk
- Ziel
- verantwortlicher Spezialist
- unterstützende Spezialisten
- Budget
- Informationsaufwand
- Risikohaltung
- Geheimhaltungsstufe

Risikohaltungen:

- vorsichtig
- ausgewogen
- aggressiv

Ergebnisse:

- kritischer Erfolg
- Erfolg
- Teilerfolg

- Fehlschlag
- kritischer Fehlschlag

Operationen müssen unterschiedliche Systeme verändern können:

- Ressourcen
- Einfluss
- Informationen
- Stress
- Loyalität
- Ermittlungsdruck
- öffentliche Aufmerksamkeit
- Bezirkswerte
- Beziehungen

Verwende reproduzierbare serverseitige Zufallswerte.

Speichere keine manipulierbaren Zufallswerte im Client.

14. Informationen und Fog of War

Spieler dürfen Gegnerwerte nicht vollständig sehen.

Informationsberichte besitzen:

- Titel
- Zusammenfassung
- Ziel
- sichtbares Vertrauensniveau
- tatsächliche, verborgene Genauigkeit
- Quelle
- Beobachtungszeit
- Ablaufdatum
- Status

Status:

- neu
- geprüft
- veraltet
- widersprochen
- bestätigt
- archiviert

Unsichere Informationen müssen in der UI sichtbar als unsicher gekennzeichnet sein.

Implementiere einen interaktiven Netzwerkgraphen aus:

- Spielern
- Spezialisten

- Unternehmen
- Organisationen
- Verträgen
- Bezirken
- Informationsverbindungen

Unsichere Verbindungen erscheinen gestrichelt.

15. Ermittlungsmodell

Trenne:

- allgemeinen Ermittlungsdruck
- konkrete Beweisobjekte

Ermittlungsstufen:

1. unauffällig
2. Aufmerksamkeit
3. Beobachtung
4. Vorermittlung
5. Strukturermittlung
6. Taskforce
7. Zugriff
8. Vermögensmaßnahmen

Beweise können entstehen aus:

- Finanzauffälligkeiten
- Unternehmensverbindungen
- Operationsfehlschlägen
- öffentlichen Vorfällen
- digitalen Spuren
- Zeugenaussagen
- internen Quellen
- verbundenen Kontakten

Mögliche Spielauswirkungen:

- Warnung
- Unternehmensprüfung
- Umsatzreduzierung
- vorübergehend blockiertes Kapital
- Spezialist nicht verfügbar
- Vertragsverlust
- eingefrorenes Unternehmen
- beschlagnahmtes Unternehmen

Der Spieler sieht niemals die vollständige interne Behördenakte.

Die sichtbare Ermittlungsakte muss ausdrücklich als Schätzung gekennzeichnet sein.

16. Bezirks- und Einflussmodell

Einflussarten:

- Wirtschaft
- Straße
- Information
- Gesellschaft
- Digital

Ein Bezirk kann gleichzeitig von mehreren Spielern und Organisationen beeinflusst werden.

Einflussanteil:

eigene aktive Einflusspunkte
geteilt durch
alle aktiven Einflusspunkte einschließlich neutralem Grundwert

Kontrollstufen:

- unbekannt
- präsent
- etabliert
- einflussreich
- dominant
- strukturell kontrollierend

Die Karte darf kein einfaches Tabellenraster sein.

Erstelle eine responsive, stilisierte Stadtkarte mit klaren Bezirksflächen und umschaltbaren Einflussfiltern.

Da Vesper fiktiv ist, verwende eine eigens gestaltete schematische Karte und keine reale Stadtgeografie.

17. Spielerorganisationen

Spieler können ab definierter Fortschrittsstufe eine Organisation gründen.

MVP-Limit:

20 Mitglieder

Rollen:

- Direktor
- Stellvertretung
- Finanzleitung
- Diplomatieleitung
- Informationsleitung
- Bezirksleitung
- Mitglied
- Anwärter

Implementiere serverseitig geprüfte Rechte:

```
organization.view
organization.edit_profile
organization.manage_roles
organization.invite
organization.remove_members
organization.suspend_members
organization.disband
treasury.view
treasury.deposit
treasury.withdraw
treasury.invest
operations.view
operations.create
operations.cancel
intel.view_shared
intel.share
diplomacy.view
diplomacy.propose
diplomacy.accept
diplomacy.terminate
research.view
research.start
audit.view
```

Implementiere:

- Organisationskasse
- Rollenverwaltung
- Einladungen
- Bewerbungen
- gemeinsames Informationsarchiv
- gemeinsame Projekte
- Mitgliederprotokoll
- Organisationsstabilität

Kritische Rechte müssen eine erneute Authentifizierung verlangen.

18. Diplomatie

Implementiere:

- Nichtangriffspakt
- Informationsaustausch
- Handelskooperation
- gemeinsame Operation

Verträge besitzen:

- Parteien
- Laufzeit
- Bedingungen
- Geltungsbereiche
- Status
- Strafklauseln
- öffentlich oder geheim
- Vertragsverletzungswert

Status:

- Entwurf
- vorgeschlagen
- angenommen
- aktiv
- abgelehnt
- abgelaufen
- gekündigt
- gebrochen

Vertragsbruch muss messbare Auswirkungen auf Vertrauen und Diplomatie haben.

19. Forschung

Kategorien:

- Organisation
- Wirtschaft
- Information
- Sicherheit
- Diplomatie
- Technologie

Forschung besitzt:

- Voraussetzungen
- Kosten
- Dauer

- serverseitige Effekte
- Abschlussereignis

Maximal:

- ein persönliches Forschungsprojekt
 - ein Organisationsforschungsprojekt
-

20. Weltereignisse

Implementiere mindestens:

- Hafenstreik
- Finanzprüfung
- Datenleck
- Wirtschaftskrise
- Medienkampagne
- Technologieboom
- Großrazzia
- Arbeitskräftemangel
- Immobilienboom
- Sicherheitskrise
- Friedensinitiative
- Lieferkettenunterbrechung

Weltereignisse müssen datengetrieben sein.

Sie dürfen nicht ausschließlich als Textmeldung existieren, sondern müssen tatsächliche Spielwerte verändern.

21. Saison und Rangliste

Lokale Testsaison:

Dauer: konfigurierbar
Standard lokal: 14 Tage
Produktion: 16 Wochen

Ranglistengewichtung:

Wirtschaftliche Macht:	25 %
Bezirks- und Markteinfluss:	20 %
Organisationsstabilität:	15 %
Informationsnetzwerk:	10 %
Diplomatischer Einfluss:	10 %

Widerstandsfähigkeit:	10 %
Gesellschaftliche Wirkung:	10 %

Strafwerte:

- hoher Ermittlungsdruck
- Vertragsbrüche
- eingefrorene Unternehmen
- interne Abspaltungen
- schwere Bezirksdestabilisierung

Erstelle getrennte Ranglisten für:

- Spieler gesamt
- Organisationen gesamt
- Wirtschaft
- Einfluss
- Stabilität
- Information
- Diplomatie

22. Benutzeroberfläche

Designstil:

- Cyber-Noir
- Schwarz
- Anthrazit
- dezentes Gold
- sparsam eingesetztes Warnrot
- hohe Lesbarkeit
- keine übermäßigen Glow-Effekte
- keine unleserlichen Minischriften
- moderne Datenvisualisierung
- responsiv ab 320 Pixel Breite

Hauptnavigation:

1. Kommando
2. Stadt
3. Netzwerk
4. Unternehmen
5. Spezialisten
6. Operationen
7. Organisation
8. Diplomatie
9. Ermittlungsakte
10. Forschung
11. Nachrichten

- 12. Rangliste
- 13. Einstellungen

Implementiere alle notwendigen Zustände:

- Laden
- leer
- Fehler
- offline
- nicht berechtigt
- Wartung
- abgeschlossene Saison

Keine leeren Attrappen-Seiten.

23. Notwendige Screens

Implementiere vollständig:

- Startseite
- Registrierung
- Login
- E-Mail-Verifikation
- Passwort vergessen
- Weltenauswahl
- Charakter- und Organisationsauswahl
- Tutorial
- Kommandozentrale
- Stadtkarte
- Bezirksdetail
- Unternehmensübersicht
- Unternehmensdetail
- Unternehmen kaufen
- Spezialistenübersicht
- Rekrutierung
- Spezialistendetail
- Operationsübersicht
- Operationsplanung
- Operationsbericht
- Netzwerkgraph
- Informationsberichte
- Ermittlungsakte
- Organisation entdecken
- Organisation erstellen
- Organisationsdashboard
- Mitgliederverwaltung
- Rollenverwaltung
- Organisationskasse
- Organisationsaudit

- Diplomatie
 - Vertragseditor
 - Forschung
 - Nachrichtenfeed
 - Benachrichtigungen
 - Ranglisten
 - Profil
 - Sitzungsverwaltung
 - Datenschutz
 - Account-Export
 - Account-Löschung
 - Adminbereich
 - Moderationsbereich
-

24. Lokale Entwicklungsumgebung

Erstelle eine vollständig reproduzierbare lokale Umgebung über Docker Compose.

Benötigte Dienste:

- PostgreSQL
- Redis
- API
- Worker
- Web
- Mailpit
- MinIO oder kompatibler lokaler Objektspeicher

Optional:

- Datenbankverwaltungstool nur im Development-Profil

Ein einziger Befehl muss das Projekt lokal starten:

```
make dev
```

Zusätzlich:

```
make setup
make migrate
make seed
make test
make test-e2e
make lint
make typecheck
make security
make validate
```

```
make reset-local  
make logs  
make stop
```

Erstelle PowerShell-kompatible Alternativen.

Nach erfolgreichem Start muss die Anwendung über eine dokumentierte lokale URL erreichbar sein.

25. Lokale Demo-Daten

Erstelle einen deterministischen Seed.

Der Seed muss enthalten:

- eine aktive Testwelt
- acht Bezirke
- mehrere Unternehmen
- mehrere Weltzustände
- mindestens 20 NPC-Testprofile
- mindestens vier Testorganisationen
- aktive Verträge
- Informationsberichte
- laufende und abgeschlossene Operationen
- einen Ermittlungsfall
- Ranglistendaten
- Weltereignisse

Erstelle lokale Demo-Konten für:

- neuer Spieler
- fortgeschrittener Spieler
- Organisationsmitglied
- Organisationsdirektor
- Moderator
- Administrator

Passwörter dürfen nicht fest im Repository stehen.

Erzeuge sie beim Seed-Vorgang und schreibe sie einmalig nach:

```
.local/demo-credentials.txt
```

Diese Datei muss durch `.gitignore` ausgeschlossen sein.

26. Internationalisierung

Die Anwendung muss vom ersten Tag vollständig internationalisierbar sein.

Verwende:

- BCP-47-Sprachcodes
- ICU MessageFormat
- CLDR-basierte Datums-, Zeit-, Zahlen- und Pluralregeln
- automatische Locale-Erkennung
- manuelle Sprachauswahl
- RTL-Unterstützung
- lokalisierte Zahlen
- lokalisierte Datumswerte
- lokalisierte Zeitangaben
- lokalisierte relative Zeiten
- passende Schrift-Fallbacks
- Unicode-Unterstützung
- UTF-8 ohne Ausnahmen

Keine sichtbaren Texte dürfen direkt in Komponenten hartcodiert sein.

Die englische Sprachdatei ist die kanonische Quelle.

Deutsch wird als zweite manuell gepflegte Sprache geführt.

Sofort mitzuliefernde Locale-Struktur

Mindestens:

```
en
de
fr
es
it
pt-BR
pt-PT
nl
pl
cs
sk
hu
ro
bg
el
tr
ru
uk
ar
```

```
he
fa
ur
hi
bn
ta
te
th
vi
id
ms
zh-CN
zh-TW
ja
ko
sv
no
da
fi
et
lv
lt
sl
hr
sr
sw
af
```

Übersetzungspipeline

Implementiere einen providerunabhängigen Übersetzungsadapter.

Unterstütze:

```
TRANSLATION_PROVIDER=disabled
TRANSLATION_PROVIDER=provider_a
TRANSLATION_PROVIDER=provider_b
TRANSLATION_PROVIDER=local
```

Keine Anbieterzugangsdaten im Code.

Erstelle folgende Befehle:

```
pnpm i18n:extract
pnpm i18n:validate
pnpm i18n:pseudo
pnpm i18n:translate --locale=fr
```



```
pnpm i18n:translate --all
pnpm i18n:report
```

Die Pipeline muss:

1. neue Schlüssel erkennen,
2. bestehende Übersetzungen bewahren,
3. fehlende Schlüssel übersetzen,
4. Platzhalter unverändert halten,
5. ICU-Syntax validieren,
6. HTML- oder Script-Injektion verhindern,
7. Übersetzungen als maschinell erzeugt markieren,
8. einen Review-Bericht erstellen,
9. bei Fehlern auf Englisch zurückfallen.

Wichtige Realitätsgrenze

Technische Sprachunterstützung und sofortiger Fallback müssen für alle konfigurierten Sprachen funktionieren.

Behaupte jedoch nicht, dass maschinell erzeugte Übersetzungen ohne menschliche Prüfung dieselbe Qualität besitzen.

Erstelle:

```
docs/localization/TRANSLATION_QUALITY.md
```

Dort müssen stehen:

- vollständig geprüfte Sprachen
- maschinell übersetzte Sprachen
- unvollständige Sprachen
- letzte Aktualisierung
- bekannte Probleme

Implementiere Pseudosprachen:

- `en-XA` für Textlängentests
- `ar-XB` für RTL-Tests

27. Barrierefreiheit

Ziel:

```
WCAG 2.2 AA
```

Implementiere:

- semantische HTML-Struktur
- Tastaturbedienung
- sichtbare Fokuszustände
- Screenreader-Beschriftungen
- ausreichende Kontraste
- reduzierte Bewegungen
- keine ausschließlich farbbasierte Statuskommunikation
- zugängliche Dialoge
- zugängliche Formulare
- Fehlermeldungsverknüpfung
- Alternativdarstellung für Graphen und Karten
- Zoom bis 200 Prozent
- Touch-Ziele mit ausreichender Größe

Führe automatisierte Accessibility-Tests durch.

28. Authentifizierung und Sicherheit

Implementiere:

- Argon2id-Passworthashing
- E-Mail-Verifikation
- Passwortzurücksetzung
- kurze Zugriffstoken
- rotierende Refresh-Tokens
- gehashte Refresh-Tokens
- Sitzungsübersicht
- Sitzungswiderruf
- optionales TOTP-2FA
- Login-Rate-Limits
- sichere Cookies oder sichere Tokenablage
- CSRF-Schutz, falls Cookies verwendet werden
- CORS-Allowlist
- Content Security Policy
- sichere HTTP-Header
- Eingabvalidierung
- Output-Encoding
- Schutz vor IDOR
- Schutz vor Mass Assignment
- Schutz vor SQL-Injection
- Schutz vor XSS
- Schutz vor Race Conditions
- Idempotency Keys
- Request-Größenlimits

Keine geheimen Daten in:

- Repository
- Git-Verlauf
- Frontend-Bundle
- Logs
- Fehlermeldungen
- Screenshots
- Seed-Dateien
- `.env.example`

Erstelle lokale Secrets automatisch und speichere sie ausschließlich in einer ignorierten Development-Datei.

Produktions-Secrets müssen über die Hostingplattform gesetzt werden.

29. Sicherheitswarnung zu vorhandenen Zugangsdaten

Suche im Repository und in lokalen Projektdateien nach versehentlich vorhandenen:

- API-Keys
- GitHub-Tokens
- Discord-Tokens
- Webhooks
- Datenbankkennwörtern
- privaten Schlüsseln
- Client-Secrets

Gib gefundene Secrets niemals vollständig aus.

Maskiere sie.

Entferne sie nicht eigenmächtig aus externen Diensten.

Erstelle bei Funden:

```
SECURITY_ROTATION_REQUIRED.md
```

Dort nur:

- Art des Secrets
- betroffene Datei
- empfohlene Rotation
- Bereinigungsstatus

Alle Zugangsdaten, die zuvor in Chats, Dokumenten oder unsicheren Dateien geteilt wurden, müssen vor einem Produktionsrelease als kompromittiert behandelt und ersetzt werden.

30. Datenintegrität

Implementiere ein Append-only-Ledger für:

- Spielerressourcen
- Organisationsressourcen
- Unternehmenserträge
- Kosten
- Erstattungen
- Korrekturen
- Beschlagnahmungen

Jeder Eintrag benötigt:

- Eigentümer
- Ressourcentyp
- Betrag
- Saldo danach
- Grund
- Referenzobjekt
- Idempotency Key
- Zeit
- Metadaten

Alte Einträge dürfen nicht geändert oder gelöscht werden.

Korrekturen erfolgen durch Gegenbuchung.

Verwende Datenbanksperren oder geeignete Concurrency-Verfahren für:

- Ressourcen
- Organisationskasse
- Unternehmenserwerb
- Operationsplätze
- Mitgliedslimits
- Vertragsannahme
- Upgradebeginn
- einmalige Startressourcen

31. Datenschutz

Implementiere technische Grundlagen für:

- Datenschutzerklärung

- Nutzungsbedingungen
- Einwilligungsverwaltung
- Datenexport
- Account-Löschung
- Sitzungsübersicht
- Datenberichtigung
- definierte Aufbewahrungsfristen
- pseudonymisierte Auditdaten
- minimierte IP-Speicherung

Keine unnötigen Analyse- oder Werbetracker.

Cookies nur, wenn technisch notwendig oder nach entsprechender Einwilligung.

Rechtstexte müssen als überprüfbare Entwürfe gekennzeichnet sein und dürfen nicht als anwaltlich geprüfte Endfassung dargestellt werden.

32. API

Erstelle eine versionierte API:

```
/api/v1
```

Bereiche:

- auth
- users
- worlds
- profiles
- resources
- districts
- businesses
- facilities
- specialists
- operations
- intelligence
- network
- investigations
- organizations
- treasury
- treaties
- research
- world-events
- news
- notifications
- rankings
- admin
- moderation

Erstelle:

- OpenAPI-Spezifikation
- generierten TypeScript-Client
- stabile Fehlercodes
- Cursor-Paginierung
- Request IDs
- Serverzeit in Antworten
- konsistente Fehlerstruktur
- Idempotency-Unterstützung

Generiere den API-Client automatisch aus der OpenAPI-Spezifikation.

Frontend und Mobile dürfen keine getrennten manuell gepflegten API-Typen besitzen.

33. WebSockets

Implementiere authentifizierte WebSocket-Ereignisse:

- Ressource geändert
- Unternehmen geändert
- Einrichtung abgeschlossen
- Operation abgeschlossen
- Informationsbericht eingetroffen
- Ermittlungsstatus verändert
- Organisation verändert
- Mitglied beigetreten
- Vertrag verändert
- Forschung abgeschlossen
- Weltereignis gestartet
- Weltereignis beendet
- Rangliste aktualisiert
- Sitzung widerrufen

WebSocket-Nachrichten sind Aktualisierungshinweise.

Nach Empfang lädt der Client den autoritativen Zustand erneut per API.

Implementiere:

- Reconnect
 - exponentiellen Backoff
 - Heartbeat
 - Ereignis-IDs
 - Deduplizierung
 - Authentifizierungserneuerung
-

34. Hintergrundaufgaben

Implementiere robuste Hintergrundprozesse für:

Minütlich

- fällige Einrichtungen
- fällige Forschungen
- fällige Operationen
- auslaufende Verträge
- Benachrichtigungen

Alle fünf Minuten

- Einflussnormalisierung
- Ermittlungsprüfung
- Spezialistenstress
- Statuskonsistenz

Stündlich

- Unternehmensabrechnung
- Marktaktualisierung
- Ranglisten
- Informationsalterung
- Ereignisauswertung

Täglich

- Gehälter
- Loyalität
- Organisationsstabilität
- Sitzungsvereinigung
- Anti-Cheat-Analyse
- Ledger-Konsistenz

Alle Jobs müssen:

- idempotent
- wiederholbar
- protokolliert
- überwacht
- gegen Parallelverarbeitung geschützt

sein.

35. Tests

Backend

Nutze:

- Unit-Tests
- Integrationstests
- API-Tests
- Datenbanktests
- Property-based Tests für zentrale Wirtschaftsformeln

Teste mindestens:

- Ressourcenbuchungen
- doppelte Requests
- parallele Requests
- Operationsresultate
- Einflussnormalisierung
- Unternehmensabrechnungen
- Ermittlungsstufen
- Verträge
- Rollenrechte
- Ranglisten
- Startressourcen
- Saisonabschluss

Web

Nutze:

- Komponenten-Tests
- Hook-Tests
- API-Mocking
- Accessibility-Tests
- Routing-Tests
- Übersetzungstests

End-to-End

Nutze Playwright.

Teste vollständig:

1. Registrierung
2. Login
3. Weltbeitritt
4. Tutorial
5. Unternehmen kaufen

6. Spezialist rekrutieren
7. Operation starten
8. Operation abschließen
9. Einfluss gewinnen
10. Organisation gründen
11. Mitglied einladen
12. Vertrag schließen
13. Forschung starten
14. Ermittlungsergebnis erhalten
15. Rangliste öffnen
16. Sprache wechseln
17. RTL-Modus
18. Logout
19. Sitzungswiderruf

Mobile

Teste:

- Navigation
- Login
- Tokenablage
- API-Client
- Sprachwechsel
- Offlinefehler
- responsive Kernansichten

Sicherheit

Führe aus:

- Dependency Audit
- Secret Scan
- statische Codeanalyse
- Backend Security Lint
- Frontend Security Scan
- Container Scan
- Berechtigungs-Tests
- IDOR-Tests
- Race-Condition-Tests

Lasttests

Erstelle Lasttests für:

- Login
- Dashboard
- Operation starten
- Ressourcenabfrage
- Ranglisten

- WebSocket-Verbindungen
- stündliche Abrechnung

Lokales Mindestziel:

- 100 simulierte Nutzer
 - keine Dateninkonsistenz
 - keine doppelten Buchungen
 - keine unkontrollierten Serverfehler
-

36. CI/CD

Erstelle GitHub-Actions-Workflows für:

Pull Requests

- Formatierung
- Lint
- Typecheck
- Unit-Tests
- Integrationstests
- Frontend-Tests
- Übersetzungsvalidierung
- Security Scan
- Build

Main Branch

- alle Prüfungen
- Produktionsbuild
- Containerbuild
- SBOM
- Container-Scan
- Release-Artefakte

Release

- versionierte Container
- Datenbankmigrationsprüfung
- Web-Build
- Mobile-Konfigurationsprüfung
- Changelog
- Release Notes

Kein Deployment, wenn:

- Tests fehlschlagen
- Migrationen fehlschlagen
- kritische Sicherheitsfunde bestehen

- Übersetzungsschlüssel fehlen
 - Ledger-Konsistenztests scheitern
-

37. AGENTS.md

Erstelle eine ausführliche `AGENTS.md`.

Sie muss Codex dauerhaft erklären:

- Projektarchitektur
- erlaubte Modulabhängigkeiten
- Testbefehle
- Sicherheitsregeln
- Datenbankregeln
- Ledgerregeln
- Übersetzungsregeln
- UI-Regeln
- Definition of Done
- verbotene Abkürzungen
- Vorgehen bei Fehlern
- Releaseverfahren

Sie muss ausdrücklich festhalten:

Eine Aufgabe ist nicht abgeschlossen, nur weil der Code kompiliert. Sie ist erst abgeschlossen, wenn relevante Tests, Sicherheitsprüfungen und Dokumentation erfolgreich sind.

38. RUN 1 – LOCAL FINAL

Führe diese Roadmap vollständig aus.

Phase 0 – Analyse und Planung

1. Projektordner prüfen.
2. Vorhandene Dateien analysieren.
3. Secrets scannen und maskiert melden.
4. Architekturentscheidung dokumentieren.
5. genaue Versionsauswahl treffen.
6. Ausführungsstatus anlegen.
7. AGENTS.md erstellen.
8. keine funktionierenden vorhandenen Teile unnötig überschreiben.

Gate: Architektur und lokale Voraussetzungen sind dokumentiert.

Phase 1 – Monorepo und Infrastruktur

1. Monorepo erzeugen.
2. Backend, Worker, Web und Mobile initialisieren.
3. gemeinsame Pakete anlegen.
4. Docker Compose erstellen.
5. PostgreSQL, Redis, Mailpit und MinIO anbinden.
6. Health Checks implementieren.
7. Makefile und PowerShell-Skripte erstellen.
8. `.env.example` ohne Secrets erstellen.
9. lokale Development-Secrets generieren.
10. CI-Grundgerüst erstellen.

Gate: Alle Container starten fehlerfrei.

Phase 2 – Datenmodell

1. Benutzer und Sitzungen.
2. Welten und Spielerprofile.
3. Ressourcen und Ledger.
4. Bezirke und Einfluss.
5. Unternehmen.
6. Einrichtungen.
7. Spezialisten.
8. Operationen.
9. Informationen.
10. Ermittlungen.
11. Organisationen.
12. Rollen und Rechte.
13. Verträge.
14. Forschung.
15. Weltereignisse.
16. Ranglisten.
17. Benachrichtigungen.
18. Audit-Logs.

Erstelle Alembic-Migrationen.

Gate: Migration auf leerer Datenbank und vollständiger Rollback-Test funktionieren.

Phase 3 – Authentifizierung und Sicherheit

1. Registrierung.
2. Verifikation.
3. Login.
4. Refresh-Rotation.
5. Logout.
6. Passwortzurücksetzung.
7. Sitzungsverwaltung.
8. 2FA-Grundlage.
9. Rate Limits.

10. sichere Header.
11. Rechteprüfungen.
12. Auditierung.

Gate: Authentifizierungs- und Berechtigungstests bestehen.

Phase 4 – Spielkern

1. Startprofil.
2. Startressourcen.
3. Unternehmen.
4. Einrichtungen.
5. Spezialisten.
6. Operationen.
7. Informationen.
8. Einfluss.
9. Ermittlungen.
10. Forschung.
11. Weltereignisse.
12. Ranglisten.

Gate: Ein vollständiger Einzelspieler-MVP-Kreislauf funktioniert.

Phase 5 – Multiplayer

1. Organisationen.
2. Mitgliedschaften.
3. Rollenrechte.
4. Organisationskasse.
5. Organisationsoperationen.
6. Informationsfreigabe.
7. Verträge.
8. Diplomatie.
9. Organisationsstabilität.

Gate: Zwei lokale Testnutzer können sicher miteinander interagieren.

Phase 6 – Web-Oberfläche

1. Designsystem.
2. Auth-Screens.
3. Weltenauswahl.
4. Tutorial.
5. Dashboard.
6. Stadtkarte.
7. Bezirksdetail.
8. Unternehmen.
9. Spezialisten.
10. Operationen.
11. Netzwerkgraph.
12. Ermittlungsakte.

13. Organisation.
14. Diplomatie.
15. Forschung.
16. Nachrichten.
17. Ranglisten.
18. Einstellungen.
19. Adminbereich.

Gate: Alle zentralen Spielabläufe funktionieren ohne direkte API-Nutzung.

Phase 7 – Internationalisierung

1. alle Texte extrahieren.
2. Englisch vollständig.
3. Deutsch vollständig.
4. konfigurierte Sprachkataloge anlegen.
5. Übersetzungsadapter.
6. Pseudosprachen.
7. RTL.
8. Übersetzungsvalidierung.
9. lokalisierte Zahlen und Zeiten.
10. Sprachumschaltung in Web und Mobile.

Gate: Keine fehlenden Übersetzungsschlüssel; Fallback funktioniert.

Phase 8 – Mobile-Grundlage

1. Expo-App.
2. Login.
3. Weltenauswahl.
4. Dashboard.
5. Unternehmen.
6. Operationen.
7. Organisation.
8. Benachrichtigungen.
9. Sprache.
10. sicherer Tokenspeicher.

Gate: Expo-App startet lokal und greift auf dieselbe API zu.

Phase 9 – Qualität

1. Unit-Tests.
2. Integrationstests.
3. End-to-End-Tests.
4. Accessibility.
5. Security Scans.
6. Lasttest.
7. Ledger-Konsistenz.
8. Datenbankmigrationstest.
9. Backup- und Restore-Test.

10. Cross-Browser-Test.

Gate: Keine kritischen Fehler und alle Pflichtprüfungen erfolgreich.

Phase 10 – Lokale Abnahme

Führe automatisch aus:

```
make clean
make setup
make migrate
make seed
make lint
make typecheck
make test
make test-e2e
make security
make validate
```

Erstelle anschließend:

```
LOCAL_FINAL_REPORT.md
```

Der Bericht enthält:

- implementierte Funktionen
- Architektur
- Startbefehle
- lokale URLs
- Testergebnisse
- Testabdeckung
- Sicherheitsprüfungen
- Demo-Konten-Pfad
- bekannte nichtkritische Einschränkungen
- Screenshots zentraler Abläufe
- Deploymentbereitschaft
- Mobile-Bereitschaft

Setze nur bei vollständiger Abnahme:

```
{
  "phase": "local_verified",
  "local_build_complete": true,
  "local_tests_passed": true,
  "security_checks_passed": true
}
```

39. RUN 1 – Verbindliche Endbedingungen

RUN 1 ist nicht fertig, bevor Folgendes funktioniert:

- `make dev`
- lokale Registrierung
- lokale E-Mail über Mailpit
- Login
- Weltbeitritt
- Tutorial
- Unternehmen kaufen
- Spezialist rekrutieren
- Operation durchführen
- Informationen erhalten
- Bezirkseinfluss verändern
- Ermittlungsdruck auslösen
- Organisation gründen
- Nutzer einladen
- Vertrag schließen
- Forschung durchführen
- Rangliste aktualisieren
- Sprache wechseln
- RTL-Darstellung
- Adminbereich
- kompletter E2E-Test
- Mobile-App lokal öffnen

Beende RUN 1 nicht bei einem roten Test.

Behebe die Ursache und führe die vollständige relevante Testsuite erneut aus.

40. RUN 2 – PRODUCTION AND MOBILE RELEASE

Starte RUN 2 nur, wenn RUN 1 validiert wurde.

Phase 11 – Produktionsaudit

1. gesamten Code erneut prüfen.
2. Abhängigkeiten aktualitäts- und sicherheitsbewerten.
3. Development-Abhängigkeiten aus Produktionsimages entfernen.
4. Secrets erneut scannen.
5. Debug-Endpunkte entfernen oder schützen.
6. CORS und CSP prüfen.
7. Rate Limits prüfen.
8. Produktionslogging prüfen.
9. Fehlermeldungen prüfen.
10. Datenschutzfunktionen prüfen.
11. Datenbankindizes analysieren.

12. langsame Queries ermitteln.
13. Lasttest wiederholen.

Gate: Produktionsaudit ohne kritische Funde.

Phase 12 – Produktionscontainer

Erstelle getrennte Container für:

- Web
- API
- Worker

Anforderungen:

- nicht als Root
- kleine Images
- Multi-Stage Builds
- Health Checks
- Read-only-Dateisystem, soweit möglich
- keine Development-Tools
- keine Secrets im Image
- sauberes Signal-Handling
- reproduzierbare Builds

Gate: Produktions-Compose startet vollständig.

Phase 13 – Railway-Vorbereitung

Bereite ein Railway-kompatibles Deployment vor:

- Web-Service
- API-Service
- Worker-Service
- PostgreSQL
- Redis
- Umgebungsvariablen
- Health Checks
- Migrationskommando
- Startkommandos
- persistente Speicheranforderungen
- Backup-Konzept
- Rollback-Konzept

Erstelle:

```
infrastructure/railway/  
├─ README.md  
├─ railway-web.json  
└─ railway-api.json
```

```
|— railway-worker.json
|— environment-template.md
```

Verwende ausschließlich Platzhalter für Secrets.

Phase 14 – Deployment-Sicherheitsgrenze

Lies:

```
ALLOW_EXTERNAL_DEPLOY
```

Wenn:

```
ALLOW_EXTERNAL_DEPLOY=false
```

dann:

- führe kein öffentliches Deployment aus
- bereite alles vollständig vor
- führe Produktionscontainer lokal aus
- erzeuge exakte Deploymentkommandos
- erstelle einen Deploymentbericht

Wenn:

```
ALLOW_EXTERNAL_DEPLOY=true
```

und alle benötigten sicheren Zugangsdaten vorhanden sind:

1. Produktionsdienste anlegen oder verbinden.
2. Datenbankmigration ausführen.
3. API deployen.
4. Worker deployen.
5. Web deployen.
6. Health Checks ausführen.
7. Smoke Tests ausführen.
8. SSL prüfen.
9. CORS prüfen.
10. WebSocket prüfen.
11. Testkonto erstellen.
12. kritischen E2E-Produktionspfad prüfen.
13. Deployment-URL dokumentieren.

Keine vorhandenen Produktionsressourcen löschen.

Keine Domain ohne ausdrückliche Konfiguration verändern.

Phase 15 – Produktionsabnahme

Prüfe live oder gegen die lokale Produktionsumgebung:

- Registrierung
- Login
- E-Mail
- Datenbankmigration
- Ressourcenbuchung
- Operation
- Worker
- WebSocket
- Organisationsrechte
- Sprachumschaltung
- RTL
- Datenschutzexport
- Account-Löschung
- Backup
- Restore
- Monitoring
- Fehlertracking

Gate: Browsergame produktionsbereit oder erfolgreich live.

41. Android-App

Die Android-App verwendet dieselbe API und dieselben Nutzerkonten.

Implementiere:

- produktive API-Konfiguration
- sichere Tokenablage
- Deep Links
- App-Links
- App-Icon-Platzhalter
- Splash Screen
- adaptive Icons
- Netzwerkstatus
- Fehlerbehandlung
- Datenschutzscreen
- Account-Löschung
- Sprachumschaltung
- RTL
- Buildprofile
- Development-Build
- Preview-Build
- Production-Build

Erzeuge:

- installierbare lokale Android-Testversion, wenn die Umgebung es erlaubt
- APK für interne Tests
- AAB für Google Play
- signierbare Release-Konfiguration
- keine privaten Signierschlüssel im Repository

Erstelle:

```
docs/mobile/ANDROID_RELEASE.md
```

Enthalten:

- benötigtes Entwicklerkonto
 - Paketname
 - Versionsschema
 - Signing
 - Datenschutzangaben
 - Store-Metadaten
 - Screenshots
 - Closed Testing
 - Production Release
 - Rollback
-

42. iOS-App

Implementiere:

- dieselben Kernfunktionen wie Android
- sichere Tokenablage
- Universal Links
- App Transport Security
- Privacy Manifest
- App-Icon-Struktur
- Splash Screen
- Buildprofile
- Development-Build
- Preview-Build
- Production-Build

Unter Windows darf für iOS eine unterstützte Cloud-Build-Pipeline verwendet werden.

Erzeuge, sofern gültige Apple-Konfiguration vorhanden ist:

- installierbaren TestFlight-Build
- produktionsfähiges iOS-Archiv

Private Apple-Zertifikate und Schlüssel dürfen nicht im Repository gespeichert werden.

Erstelle:

docs/mobile/IOS_RELEASE.md

Enthalten:

- Apple Developer Account
- Bundle Identifier
- Signing
- App Store Connect
- TestFlight
- Datenschutzangaben
- Reviewinformationen
- In-App-Accountlöschung
- Production Release
- Rollback

Eine Freigabe durch Apple oder Google darf nicht garantiert werden, da sie von externen Konten und Prüfverfahren abhängt.

43. App-Store-Metadaten

Erstelle Store-Metadaten mindestens für:

- Englisch
- Deutsch
- Französisch
- Spanisch
- Italienisch
- Portugiesisch
- Polnisch
- Türkisch
- Arabisch
- Chinesisch
- Japanisch
- Koreanisch
- Vietnamesisch

Enthalten:

- App-Name
- Kurzbeschreibung
- Langbeschreibung
- Keywords
- Altersfreigabeinformationen
- Supporttext

- Datenschutz-URL-Platzhalter
- Marketingtext
- Screenshotplan
- Release Notes

Keine falschen Versprechen wie:

- „vollständig realistisch“
- „garantiert sicher“
- „100 Prozent fehlerfrei“
- „alle Übersetzungen menschlich geprüft“

44. Releaseversionierung

Verwende Semantic Versioning.

Beispiel:

```
0.1.0 - lokaler MVP
0.2.0 - geschlossene Browser-Alpha
0.3.0 - Mobile Preview
0.5.0 - öffentliche Beta
1.0.0 - erster stabiler Saisonrelease
```

Erstelle:

- `CHANGELOG.md`
- Release Notes
- Migrationshinweise
- Rückwärtskompatibilitätsbericht
- Datenbank-Backup vor Migration
- Rollback-Anleitung

45. Observability

Implementiere:

- strukturierte Logs
- Request IDs
- Health Endpoint
- Readiness Endpoint
- Metriken
- Fehlertracking-Adapter
- Queue-Monitoring
- Datenbankverbindungsüberwachung

- Ledger-Konsistenzalarm
- Worker-Staualarm
- ungewöhnliche Ressourcenentstehung
- ungewöhnliche Loginaktivität

Keine personenbezogenen oder geheimen Daten in Logs.

46. Backup und Restore

Erstelle:

- lokales Backupskript
- lokales Restoreskript
- PostgreSQL-Backup
- Objektspeicher-Backup
- Wiederherstellungsdokumentation
- automatisierten Restore-Test mit Testdatenbank

Ein Backup gilt erst als valide, wenn ein Restore-Test erfolgreich war.

47. Abschlussprüfung

Am Ende jedes Runs:

1. `git status` prüfen.
2. Secrets scannen.
3. alle Migrationen testen.
4. alle Tests ausführen.
5. Security Scan ausführen.
6. Produktionsbuild erzeugen.
7. Dokumentation aktualisieren.
8. Statusdatei aktualisieren.
9. Abschlussbericht schreiben.

Verbotene Abschlusssaussage

Sage nicht einfach:

„Das Projekt ist fertig.“

Gib stattdessen Beweise aus:

- ausgeführte Befehle
- Exit Codes
- Testanzahl
- fehlgeschlagene Tests
- behobene Fehler

- Build-Artefakte
 - lokale URLs
 - Produktionsstatus
 - bekannte Einschränkungen
 - nächster konkreter Release-Schritt
-

48. Definition of Done

Eine Funktion ist nur fertig, wenn:

- Backend implementiert
- Frontend implementiert
- mobile Auswirkungen berücksichtigt
- autorisiert
- validiert
- protokolliert
- getestet
- internationalisiert
- barrierearm
- dokumentiert
- ohne kritische Sicherheitsfunde

ist.

Kompilieren allein reicht nicht.

49. Verbindliche Qualitätsregeln

- Keine kritischen TODOs.
 - Keine Fake-API.
 - Keine Mock-Daten außerhalb klarer Development-Seeds.
 - Keine clientseitige Autorität.
 - Keine ungeschützten Adminendpunkte.
 - Keine hartcodierten Secrets.
 - Keine hartcodierten UI-Texte.
 - Keine doppelten Wirtschaftsbuchungen.
 - Keine fehlenden Datenbankmigrationen.
 - Keine still ignorierten Exceptions.
 - Keine leeren Catch-Blöcke.
 - Keine ungetesteten kritischen Rechte.
 - Keine fehlerhaften RTL-Layouts.
 - Keine Veröffentlichung bei roten Tests.
 - Keine Behauptung menschlich geprüfter maschineller Übersetzungen.
 - Keine Verwendung realer krimineller Gruppen oder Methoden.
-

50. Erwartete Abschlussdateien

Nach RUN 1:

```
LOCAL_FINAL_REPORT.md
TEST_REPORT.md
SECURITY_REPORT.md
ACCESSIBILITY_REPORT.md
LOCALIZATION_REPORT.md
ARCHITECTURE_REPORT.md
```

Nach RUN 2:

```
PRODUCTION_RELEASE_REPORT.md
DEPLOYMENT_REPORT.md
ANDROID_RELEASE_REPORT.md
IOS_RELEASE_REPORT.md
STORE_SUBMISSION_CHECKLIST.md
POST_RELEASE_MONITORING.md
```

51. Starte jetzt

Beginne sofort mit der Umgebungs- und Repository-Prüfung.

Danach:

1. Bestimme anhand von `.project/execution-state.json` den aktuellen Run.
2. Erstelle oder aktualisiere `AGENTS.md`.
3. Führe die passende Roadmap vollständig aus.
4. Arbeite Fehler eigenständig ab.
5. Wiederhole Tests nach jeder relevanten Korrektur.
6. Stoppe nicht bei einem bloß kompilierenden Zustand.
7. Liefere ein tatsächlich startbares und nachweisbar getestetes Ergebnis.

Das Endziel lautet:

Ein lokal vollständig spielbares und getestetes SHADOWGRID-Browsergame, das im zweiten Lauf sicher als produktive Webanwendung bereitgestellt sowie als gemeinsame Android- und iOS-App gebaut werden kann.